PRÁCTICA AUTÓNOMA · SESIÓN 19

Ciclos y menús de la UA1 en C

while, do … while, for anidados y switch

Módulo	CSTI12010 Diseño de algoritmos
Unidad de aprendizaje	UA2 · Empleo de lenguajes de programación en el paradigma de programación estructurada
Capacidad	2.2 Emplear estructuras de control del lenguaje de programación en la resolución de problemas según las especificaciones técnicas.
Duración	55 minutos · trabajo individual
Herramienta	VS Code, gcc (MSYS2) y git
Persona facilitadora	Giovanni Antonio Coto Calderón
Persona estudiante	_____
Fecha	____/____/_____

Antes de empezar

Con esta práctica usted demuestra que puede:

- Procesar datos con un centinela, un contador y un acumulador.
- Validar entradas con do … while.
- Dibujar patrones con for anidados.
- Construir un menú con switch dentro de un ciclo.

Instrucciones generales

- Trabajo **individual**. Puede consultar el documento de conceptos del día y sus apuntes de la práctica guiada. Si pide ayuda, que sea para entender, no para copiar (valor: honradez).
- Guarde todo en la carpeta **UA2/S19/** de su portafolio, con los nombres indicados (sin tildes, sin ñ y sin espacios).
- Al terminar, complete la autoevaluación del final.

Recordatorio

PSeInt	C
`Mientras c Hacer … FinMientras`	`while (c) { … }`
`Repetir … Hasta Que c`	`do { … } while (condición contraria);`
`Para i <- 1 Hasta n Con Paso 1`	`for (i = 1; i <= n; i++) { … }`

PSeInt	C
`Segun x Hacer 1: … De Otro Modo: …`	`switch (x) { case 1: … break; default: … }`
`x <- x + 1  /  t <- t + v`	`x++;  /  t += v;`

En cada programa escriba un **comentario por instrucción** con lo que hace y el resultado esperado. Si un programa no termina, deténgalo con **Ctrl + C**.

Ejercicios

Ejercicio 1 · PlanillaSemanal (sesión 12) → planilla.c (15 min)

Qué se logra: Calcular la planilla de un grupo de personas sin saber de antemano cuántas son: el -1 indica el final.

Elementos que se practican: while con centinela y lectura previa, if … else dentro del ciclo, dos contadores y un acumulador, constantes con #define.

Resultado esperado: 40 h a 2500 → Pago: 100000.00; 50 h a 3000 → Pago: 153000.00 (2 horas extra a 1.5); al final Personas: 2, Con horas extra: 1 y Total de la planilla: 253000.00.

PlanillaSemanal.psc

```
Algoritmo PlanillaSemanal
    // Pago semanal con horas extra de varias personas colaboradoras
    Definir cantidad, i, conExtra Como Entero;
    Definir nombre Como Caracter;
    Definir horas, tarifa, horasExtra, pago, totalPlanilla Como Real;
    totalPlanilla <- 0;
    conExtra <- 0;
    Escribir "Cantidad de personas colaboradoras:";
    Leer cantidad;
    Para i <- 1 Hasta cantidad Con Paso 1 Hacer
        Escribir "Nombre:";
        Leer nombre;
        Repetir
            Escribir "Horas de la semana (0 a 72):";
            Leer horas;
            Si horas < 0 O horas > 72 Entonces
                Escribir "Dato inválido, intente de nuevo";
            FinSi
        Hasta Que horas >= 0 Y horas <= 72
        Escribir "Tarifa por hora:";
        Leer tarifa;
        Si horas > 48 Entonces
            horasExtra <- horas - 48;
            pago <- 48 * tarifa + horasExtra * tarifa * 1.5;
            conExtra <- conExtra + 1;
        SiNo
            pago <- horas * tarifa;
        FinSi
        Escribir nombre, ": ", pago;
        totalPlanilla <- totalPlanilla + pago;
    FinPara
    Escribir "Total de la planilla: ", totalPlanilla;
    Escribir "Personas con horas extra: ", conExtra;
FinAlgoritmo
```

Salida real en PSeInt · datos de prueba: 2, Ana, 40, 2500, Luis, 50, 3000

```
Cantidad de personas colaboradoras:
> 2
Nombre:
> Ana
Horas de la semana (0 a 72):
> 40
Tarifa por hora:
> 2500
Ana: 100000
Nombre:
> Luis
Horas de la semana (0 a 72):
> 50
Tarifa por hora:
> 3000
Luis: 153000
Total de la planilla: 253000
Personas con horas extra: 1
```

En C todavía no leemos nombres completos (sesión 20). Cambie el **Para** por un **while con centinela**: lea las horas de cada persona hasta digitar **-1**. Use **#define LIMITE 48**, **#define RECARGO 1.5** y **#define CENTINELA -1**. Al final muestre cuántas personas, cuántas con horas extra y el total.

Ejecución esperada en la terminal

```
Horas de la semana (-1 para terminar): 40
Tarifa por hora: 2500
Pago: 100000.00
Horas de la semana (-1 para terminar): 50
Tarifa por hora: 3000
Pago: 153000.00
Horas de la semana (-1 para terminar): -1
Personas: 2
Con horas extra: 1
Total de la planilla: 253000.00
```

Ejercicio 2 · PromedioN (sesión 7) → promedio.c (12 min)

Qué se logra: Promediar notas sin aceptar datos imposibles: ni una cantidad fuera de 1 a 10 ni notas fuera de 0 a 100.

Elementos que se practican: for para recorrer las notas y do … while para validar (dos veces: la cantidad y cada nota); acumulador.

Resultado esperado: Rechaza la cantidad 0 y la nota 120; con 80, 75 y 90 muestra Suma: 245.00 y Promedio: 81.67.

PromedioN.psc

```
Algoritmo PromedioN
    // Promedio de una cantidad variable de notas
    Definir cantidad, i Como Entero;
    Definir nota, suma, promedio Como Real;
    suma <- 0;
    Escribir "¿Cuántas notas va a digitar?";
    Leer cantidad;
    Para i <- 1 Hasta cantidad Con Paso 1 Hacer
        Escribir "Nota ", i, ":";
        Leer nota;
        suma <- suma + nota;
    FinPara
    promedio <- suma / cantidad;
    Escribir "Suma: ", suma;
    Escribir "Promedio: ", promedio;
FinAlgoritmo
```

Salida real en PSeInt · datos de prueba: 3, 80, 75, 90

```
¿Cuántas notas va a digitar?
> 3
Nota 1:
> 80
Nota 2:
> 75
Nota 3:
> 90
Suma: 245
Promedio: 81.6666666667
```

Traduzca con **for** y agregue dos validaciones con **do … while**: la cantidad debe estar entre 1 y 10, y cada nota entre 0 y 100. Las notas inválidas no se suman.

Ejecución esperada en la terminal

```
Cuantas notas (1 a 10): 0
Cuantas notas (1 a 10): 3
Nota 1 (0 a 100): 80
Nota 2 (0 a 100): 120
Nota 2 (0 a 100): 75
Nota 3 (0 a 100): 90
Suma: 245.00
Promedio: 81.67
```

Ejercicio 3 · Patrones con for anidados → patrones.c (10 min)

Qué se logra: Dibujar figuras en la terminal para entender cómo un ciclo interno depende del externo.

Elementos que se practican: for anidados donde el límite del interno es la fila del externo (col <= fila).

Resultado esperado: Con 4 filas: un triángulo de asteriscos (*, **, ***, ****) y otro de números (1, 12, 123, 1234).

Lea la cantidad de filas y dibuje los dos patrones. Pista: el ciclo interno da tantas vueltas como el **número de la fila**.

Ejecución esperada en la terminal

```
Cantidad de filas: 4
*
**
***
****
1
12
123
1234
```

Ejercicio 4 · AreaFiguras (sesión 11) → areas.c (13 min)

Qué se logra: Calcular varias áreas en una misma ejecución, eligiendo la figura en un menú.

Elementos que se practican: switch con case, break y default dentro de un do … while; constante PI con #define.

Resultado esperado: Cuadrado de lado 4 → Area: 16.00; círculo de radio 2 → Area: 12.57; opción 7 → Opcion invalida; 0 → Hasta luego.

AreaFiguras.psc

```
Algoritmo AreaFiguras
    Definir opcion Como Entero;
    Definir lado, base, altura, radio Como Real;
    Escribir "1. Cuadrado";
    Escribir "2. Rectángulo";
    Escribir "3. Círculo";
    Escribir "Opción:";
    Leer opcion;
    Segun opcion Hacer
        1:
            Escribir "Lado:";
            Leer lado;
            Escribir "Área: ", lado ^ 2;
        2:
            Escribir "Base:";
            Leer base;
            Escribir "Altura:";
            Leer altura;
            Escribir "Área: ", base * altura;
        3:
            Escribir "Radio:";
            Leer radio;
            Escribir "Área: ", PI * radio ^ 2;
        De Otro Modo:
            Escribir "Opción inválida";
    FinSegun
FinAlgoritmo
```

Salida real en PSeInt · datos de prueba: 1, 4

```
1. Cuadrado
2. Rectángulo
3. Círculo
Opción:
> 1
Lado:
> 4
Área: 16
```

Traduzca el **Según** a **switch** y agregue la opción **0. Salir** dentro de un **do … while**, para calcular varias áreas sin volver a ejecutar el programa. Recuerde: C no tiene **^** (lado * lado) y PI se define con **#define**.

Ejecución esperada en la terminal

```
1. Cuadrado  2. Rectangulo  3. Circulo  0. Salir
Opcion: 1
Lado: 4
Area: 16.00

1. Cuadrado  2. Rectangulo  3. Circulo  0. Salir
Opcion: 3
Radio: 2
Area: 12.57

1. Cuadrado  2. Rectangulo  3. Circulo  0. Salir
Opcion: 7
Opcion invalida

1. Cuadrado  2. Rectangulo  3. Circulo  0. Salir
Opcion: 0
Hasta luego
```

Reto opcional · El título subrayado (sesión 17)

Resuelva el reto de la sesión 17: guarde en **largo** lo que devuelve **printf("COMPROBANTE DE PAGO\n")** menos 1 y use un **for** para imprimir esa misma cantidad de guiones debajo del título.

Evidencia en el portafolio

Complete **UA2/S19/README.md** (archivos, una prueba de ejecución, lo que aprendió, dificultades y autoevaluación), marque la **S19** con ☑ en el registro de evidencias y agregue la fila del día en la **bitácora**.

Commits: uno por ejercicio (S19: agrega planilla.c, …), otro para el README y **git push** al final.

Entrega

Archivo	Contenido
`UA2/S19/planilla.c`	Ejercicio 1.

Archivo	Contenido
`UA2/S19/promedio.c`	Ejercicio 2.
`UA2/S19/patrones.c`	Ejercicio 3.
`UA2/S19/areas.c`	Ejercicio 4.
`UA2/S19/README.md`	Evidencia de la sesión completa.
`Historial de commits`	Un commit por ejercicio y git push.

Autoevaluación

Marque con X según corresponda. Sea honesto: esta información sirve para que usted y la persona facilitadora sepan qué reforzar.

Indicador	Sí	Aún no
Elijo while, do … while o for según el problema.		
Escribo la condición contraria al traducir Repetir … Hasta Que.		
Inicializo contadores y acumuladores en 0.		
Uso lectura previa con un centinela.		
Uso for anidados y sé cuántas veces se ejecuta el ciclo interno.		
Cierro cada case con break e incluyo default.		
Mis programas compilan con -Wall -Wextra sin advertencias.		